

1.What is Git?

- Define Git as a version control system
- Explain its purpose (tracking changes, collaboration, etc.)

Git is a distributed version control system that allows developers to record, manage, and track changes in their code or files.

Purpose of Git

Tracking Changes

Git keeps a history of every change made to your files.

You can go back to any previous version if something breaks.

Collaboration

Multiple people can work on the same project at the same time.

Git helps merge changes from different team members without conflicts.

Version Management

You can create different versions (called branches) of your project.

Useful for testing new features without affecting the main code.

Backup & Safety

Your code is safely stored and can be recovered anytime.

Mistakes can be undone easily.

Distributed System

Every user has a full copy of the project history.

Work can continue even without internet

2. What is Version Control?

- Explain the concept of version control
- Importance in software development

Version Control is a system that helps you track, manage, and store changes made to files (especially code) over time.

Concept of Version Control

The main idea behind version control is:

Tracking Changes: Every modification in the file is recorded

Version History: You can view previous versions

Branching: Work on different features separately without affecting the main code

Merging: Combine different changes into one final version

Collaboration: Multiple people can work on the same project without conflict

Importance in Software Development

Version control is very important because:

1. Prevents Data Loss

If something breaks, you can restore an older version

2. Enables Teamwork

Many developers can work together on the same project

3. Tracks History

You can see what changes were made and why

4. Supports Experimentation

Developers can try new ideas without affecting the main project

5. Improves Productivity

Saves time and avoids confusion in large projects

3. What is GitHub?

- Define GitHub as a platform for hosting repositories

- Mention its role in collaboration

GitHub is an online platform used to host and manage repositories (projects) that use version control.

Definition

GitHub is a platform where developers:

Upload their projects (repositories)

Track changes using Git

Access their code from anywhere

Role in Collaboration

GitHub plays a major role in teamwork and collaboration:

1. Multiple Developers Can Work Together

Many people can work on the same project at the same time

2. Pull Requests

Developers can suggest changes

Team members can review before accepting

3. Code Review & Discussion

You can comment on code

Discuss improvements directly in the project

4. Branching & Merging

Work on new features separately

Merge them safely into the main project

5. Centralized Project Storage

All project files are stored in one place (repository)

Easy to manage and share

4. Difference Between Git and GitHub

- Write at least 3–5 clear differences

Git and GitHub are related but different:

Git is a version control system, while GitHub is a platform for hosting Git repositories.

Git works on your local computer and can be used offline, but GitHub is an online service that requires internet.

Git is used to track changes and manage versions of code, whereas GitHub is used to store, share, and manage code online.

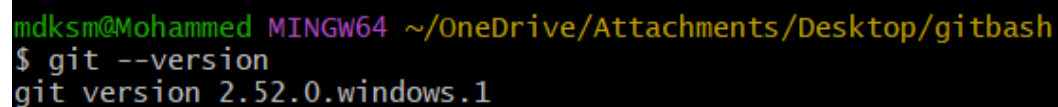
Git has limited collaboration features, but GitHub provides easy collaboration tools like pull requests and code reviews.

Git is a software tool, while GitHub is a web-based platform built on Git.

basic command and their description

`git --version`

Checks the installed Git version on



```
mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ git --version
git version 2.52.0.windows.1
```

`git config --global user.name "Your Name"`

Sets the global username for Git commits.

`git config --global user.email email@gmail.com`

Sets the global email for Git commits.

`git config user.name`

Displays the configured Git username.

`git config user.email`

Displays the configured Git email.

`git config --list`

Shows all Git configuration settings.

 MINGW64:/c/Users/mdksm

mdksm@Mohammed MINGW64 ~

\$ git --version

git version 2.52.0.windows.1

mdksm@Mohammed MINGW64 ~

\$ git config --global user.name "khalandar"

mdksm@Mohammed MINGW64 ~

\$ git config --global user.email "khalandar369@outlook.com"

mdksm@Mohammed MINGW64 ~

\$ git config --list

diff.astextplain.textconv=astextplain

filter.lfs.clean=git-lfs clean -- %f

filter.lfs.smudge=git-lfs smudge -- %f

filter.lfs.process=git-lfs filter-process

filter.lfs.required=true

http.sslbackend=schannel

core.autocrlf=true

core.fscache=true

core.symlinks=false

pull.rebase=false

credential.helper=manager

credential.https://dev.azure.com.usehttppath=true

init.defaultbranch=master

user.email=khalandar369@outlook.com

user.name=khalandar

```
mdksm@Mohammed MINGW64 ~  
$ git config user.name  
khalandar
```

```
mdksm@Mohammed MINGW64 ~  
$ git config user.email  
khalandar369@outlook.com
```

git help

Displays help/manual for Git commands.

```

mdksm@Mohammed MINGW64 ~
$ git help
usage: git [-v | --version] [-h | --help] [-C <path>] [-c <name>=<value>]
          [--exec-path[=<path>]] [--html-path] [--man-path] [--info-path]
          [-p | --paginate | -P | --no-pager] [--no-replace-objects] [--no-lazy-fetch]
          [--no-optional-locks] [--no-advice] [--bare] [--git-dir=<path>]
          [--work-tree=<path>] [--namespace=<name>] [--config-env=<name>=<envvar>]
          <command> [<args>]

These are common Git commands used in various situations:

start a working area (see also: git help tutorial)
  clone      Clone a repository into a new directory
  init       Create an empty Git repository or reinitialize an existing one

work on the current change (see also: git help everyday)
  add        Add file contents to the index
  mv         Move or rename a file, a directory, or a symlink
  restore    Restore working tree files
  rm         Remove files from the working tree and from the index

examine the history and state (see also: git help revisions)
  bisect    Use binary search to find the commit that introduced a bug
  diff      Show changes between commits, commit and working tree, etc
  grep      Print lines matching a pattern
  log       Show commit logs
  show      Show various types of objects
  status    Show the working tree status

grow, mark and tweak your common history
  backfill  Download missing objects in a partial clone
  branch    List, create, or delete branches
  commit    Record changes to the repository
  merge     Join two or more development histories together
  rebase    Reapply commits on top of another base tip
  reset     Reset current HEAD to the specified state
  switch    Switch branches
  tag       Create, list, delete or verify tags

collaborate (see also: git help workflows)
  fetch     Download objects and refs from another repository
  pull      Fetch from and integrate with another repository or a local branch
  push      Update remote refs along with associated objects

'git help -a' and 'git help -g' list available subcommands and some
concept guides. See 'git help <command>' or 'git help <concept>'
to read about a specific subcommand or concept.
See 'git help git' for an overview of the system.

```

pwd

Shows the current working directory (present folder location).

cd

folder_name/location Changes the current directory to another folder.

Ls

Shows the current working directory (present folder location).

ls -l

Changes the current directory to another folder.

ls -a

Lists all files including hidden files (files starting with .)

`ls -d */`

Lists only directories (folders) in the current location.

```
mdksm@Mohammed MINGW64 ~/OneDrive/Music
$ cd C:/Users/mdksm/OneDrive/Music

mdksm@Mohammed MINGW64 ~/OneDrive/Music
$ ls
Playlists/  desktop.ini

mdksm@Mohammed MINGW64 ~/OneDrive/Music
$ ls -l
Playlists/
desktop.ini

mdksm@Mohammed MINGW64 ~/OneDrive/Music
$ ls -a
./  ../  Playlists/  desktop.ini

mdksm@Mohammed MINGW64 ~/OneDrive/Music
$ ls -d */
Playlists//
```

```
mdksm@Mohammed MINGW64 ~/OneDrive/Music
$ ls
Playlists/  file1.txt  file2.txt  file4.txt  file6.txt  file8.txt  python1/  python2/  python4/  python6/  python8/
desktop.ini  file10.txt  file3.txt  file5.txt  file7.txt  file9.txt  python10/  python3/  python5/  python7/  python9/
```

`touch filename`

Creates a new empty file.

`touch file1 file2 file3`

Creates multiple files at once. `rm filename` Deletes a file.

`mkdir folder_name`

Creates a new folder.

`mkdir folder1 folder2 folder3`

Creates multiple folders at once.

`rmdir folder_name`

Deletes an empty folder.

`rm -r foldername`

Deletes a folder and its contents (non-empty folder).

```
mdksm@Mohammed MINGW64 ~/OneDrive/Music
$ touch file1.txt

mdksm@Mohammed MINGW64 ~/OneDrive/Music
$ touch file2.txt file3.txt file4.txt file5.txt file6.txt file7.txt file8.txt file9.txt file10.txt

mdksm@Mohammed MINGW64 ~/OneDrive/Music
$ mkdir python1 python2 python3 python4 python5 python6 python7 python8 python9 python10
```

```
mdksm@Mohammed MINGW64 ~/OneDrive/Music
$ rm file1.txt file2.txt

mdksm@Mohammed MINGW64 ~/OneDrive/Music
$ ls
Playlists/ file10.txt file4.txt file6.txt file8.txt python1/ python2/ python4/ python6/ python8/
desktop.ini file3.txt file5.txt file7.txt file9.txt python10/ python3/ python5/ python7/ python9/

mdksm@Mohammed MINGW64 ~/OneDrive/Music
$ rmdir python10 python5

mdksm@Mohammed MINGW64 ~/OneDrive/Music
$ ls
Playlists/ desktop.ini file10.txt file3.txt file4.txt file5.txt file6.txt file7.txt file8.txt file9.txt python1/ python2/ python3/ python4/ python6/ python7/ python8/ python9/

mdksm@Mohammed MINGW64 ~/OneDrive/Music
$ |
```

cat filename

Displays the contents of a file in the terminal.

echo "text-content" > filename

Writes text to a file. If the file does not exist, it will be created.

echo "text" >> filename

Appends text to an existing file without deleting previous content.

```
mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ echo "practicing git" > gitfile.txt

mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ cat gitfile.txt
practicing git
```

```
mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ echo "git commands2" >>gitfile.txt

mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ cat gitfile.txt
practicing git
git commands2
```

`cat > filename`

Creates a file and allows us to type text directly in the terminal.

Steps:

`cat > file.txt`

Type your text here Press Ctrl + D to save and exit

`cat >> filename`

Adds text to an existing file without deleting previous content.

Steps:

`cat >> file.txt`

Type additional text Press Ctrl + D to save

```
git commands for creating appending and deleting files using echo and cat
mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ cat >> gitfile.txt
{gitcommand for creating appending and deleting files using echo and cat}
mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ cat gitfile.txt
git commands for creating appending and deleting files using echo and{gitgit command for creating appending and deleting files using echo and cat}
mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ |
```

`cat file1.txt > file2.txt`

Copies the content of file1.txt into file2.txt.

```
mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ echo "git" > gitfile2.txt

mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ ls
gitfile.txt  gitfile2.txt

mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ cat gitfile2.txt
git

mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ cat gitfile.txt > gitfile2.txt

mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ cat gitfile2.txt
git commands for creating appending and deleting files using echo and{gitgit command for creating appending and deleting files using echo and cat}

mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$
```

nano file.txt

Opens the Nano text editor to create or edit a file.

Opens Steps in Nano Editor:

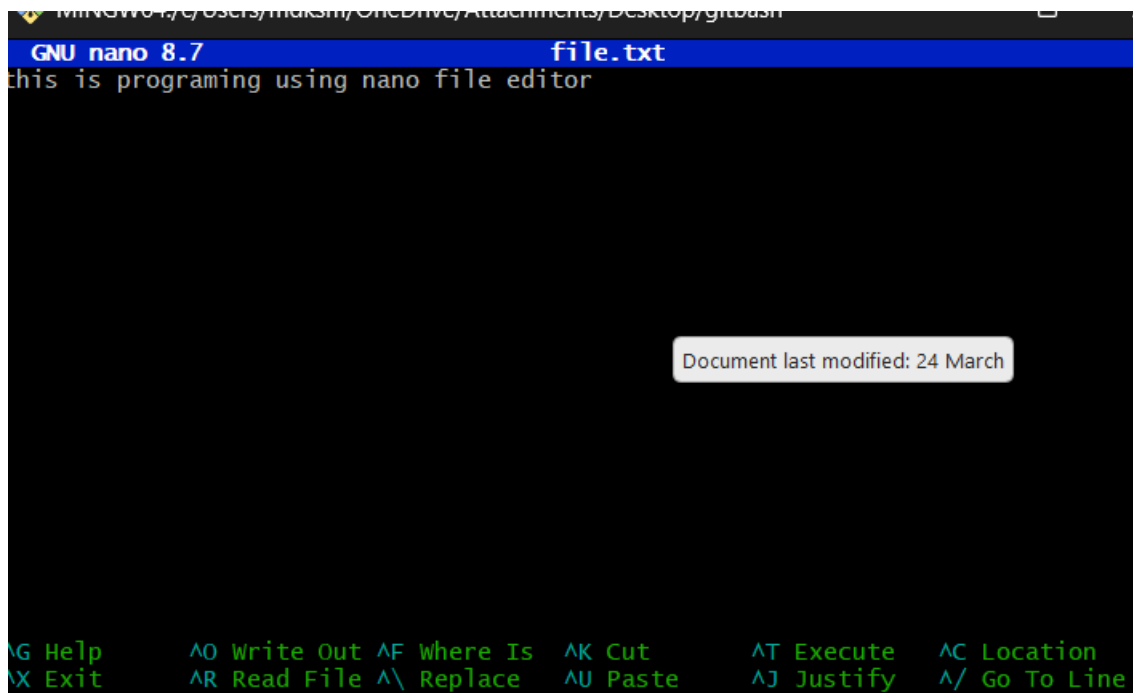
Save the file Ctrl + O

Exit the editor Ctrl + X

```
mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ nano file.txt
```

```
mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ ls
file.txt      gitfile2.txt  labfile.pdf.txt
gitfile.txt   khalandar.txt  mohammedkhalandar.txt

mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ cat file.txt
this is programing using nano file editor
```



```
GNU nano 8.7 file.txt
this is programing using nano file editor

Document last modified: 24 March

^G Help      ^O Write Out ^F Where Is  ^K Cut       ^T Execute   ^C Location
^X Exit      ^R Read File ^\ Replace   ^U Paste     ^J Justify   ^_ Go To Line
```

1. Check the default editor via

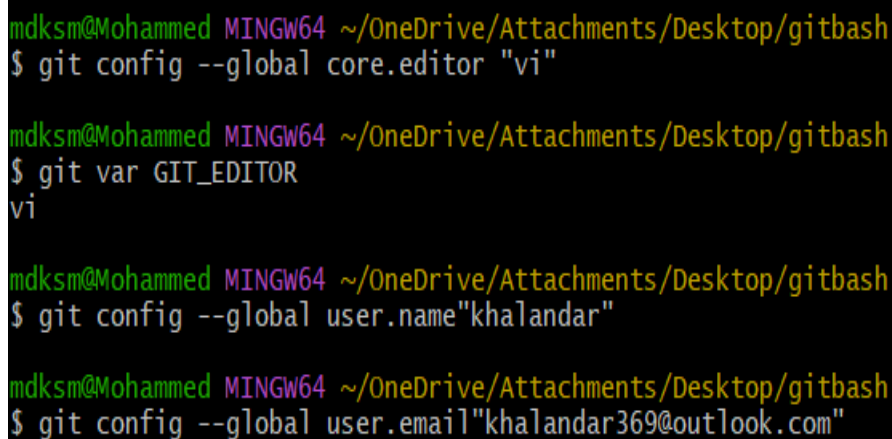
Git git var GIT_EDITOR

2. Set vi editor as the default editor for Git Bash

git config --global core.editor "vi"

3. Opening a File

vi filename.txt

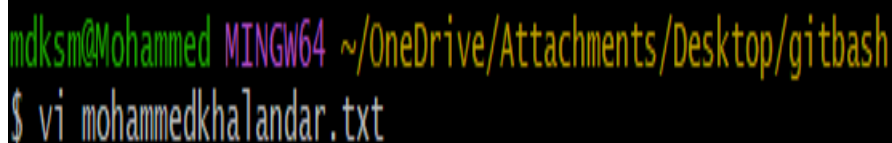


```
mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ git config --global core.editor "vi"

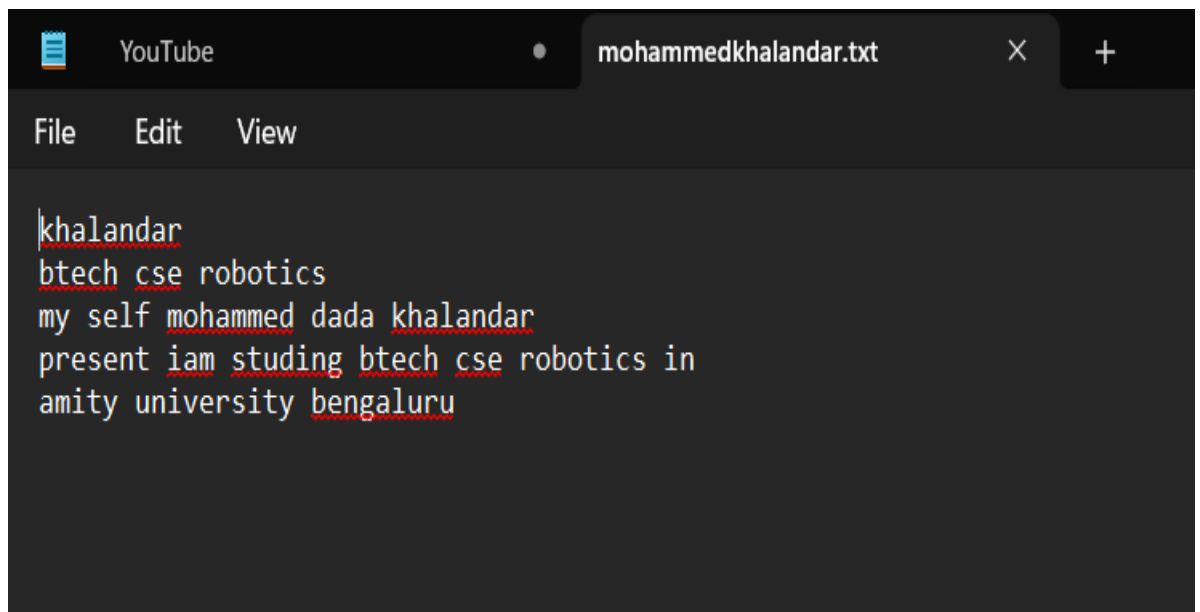
mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ git var GIT_EDITOR
vi

mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ git config --global user.name "khalandar"

mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ git config --global user.email "khalandar369@outlook.com"
```



```
mdksm@Mohammed MINGW64 ~/OneDrive/Attachments/Desktop/gitbash
$ vi mohammedkhalandar.txt
```



The screenshot shows a text editor window with a dark theme. The title bar at the top says 'mohammedkhalandar.txt'. Below the title bar is a menu bar with 'File', 'Edit', and 'View'. The main text area contains the following text: 'khalandar', 'btech cse robotics', 'my self mohammed dada khalandar', 'present iam studing btech cse robotics in', and 'amity university bengaluru'. Each word in the text is underlined with a red dotted line.

i

Inserts text before the current cursor.

I

Inserts text at the beginning of line.

a

Insert text after the current cursor.

A

Insert text at the end of the current line.

o

Creates a new line for text entry below cursor location and switches to insert mode.

O

Creates a new line for text entry above cursor location and switches to insert mode.

:w

Save File

:q

Quit

:wq

Save and quit

:x

Save and quit

:q!

Quit without saving.

x

Delete character dd Delete current line

d\$

Delete from cursor to end of line

d0

Delete from cursor to start of line

:q!

Quit without saving.

y

Delete character

p

Delete current line

P

Delete from cursor to end of line

d0

Delete from cursor to start of line

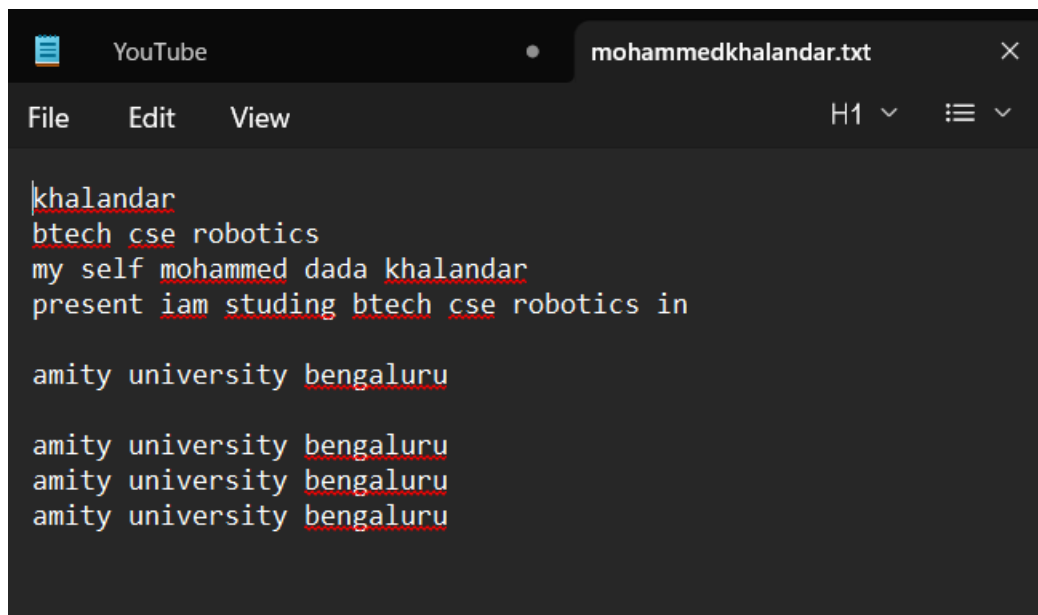

```
MINGW64:/c/Users/mdksm/OneDrive/Attachments/Desktop/gitbash
khalandar
btech cse robotics
my self mohammed dada khalandar
present iam studing btech cse robotics in

amity university bengaluru
this is vi| file
~
```

```
YouTube mohammedkhalandar.txt
File Edit View
khalandar
btech cse robotics
my self mohammed dada khalandar
present iam studing btech cse robotics in

amity university bengaluru
this is vi file
```

[illegible]



```
khalandar
btech cse robotics
my self mohammed dada khalandar
present iam studing btech cse robotics in

amity university bengaluru

amity university bengaluru
amity university bengaluru
amity university bengaluru
```